



CompSci 230

Object Oriented Software Development

Arrays



Review Quizzes

▶ What will be returned from the following method?

- ▶ 18.0
- ▶ 18 (as an integer)
- ▶ 8.0
- ▶ This is an error

```
public static int methodA() {  
    double a = 8.5 + 9.5;  
    return a;  
}
```

▶ Which of the following would be a valid method call for the following method?

- ▶ showProduct("5", "40");
- ▶ showProduct(10.0, 4.6);
- ▶ showProduct(3.3, 55);
- ▶ showProduct(10, 4.5);

```
public static void showProduct(double num1, int num2)  
{  
    double product;  
    product = num1 * num2;  
    System.out.println("The product is " + product);  
}
```



Agenda & Reading

► Topics:

► Arrays

- Multidimensional Arrays
- Arrays of Objects

► ArrayLists

- ArrayList with Generics
- Wrapper Classes

► ArrayLists Vs Arrays

► Reading

► Java how to program Late objects version (D & D)

- Chapter 6

► The Java Tutorial

► Arrays

- <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html>
- http://www.tutorialspoint.com/java/java_arrays.htm

► ArrayLists

- http://www.tutorialspoint.com/java/java_arraylist_class.htm



1.Introduction

- ▶ Data structures — collections of related data items.
- ▶ Array objects — data structures consisting of related data items of the **same type**.
 - ▶ Make it convenient to process related groups of values.
 - ▶ Remain the same length once they're created.
- ▶ Enhanced **for** statement : allows a program to access the data in an array more easily than does the counter-controlled for statement.
- ▶ Use **variable-length argument lists** to create methods that can be called with varying numbers of arguments.
- ▶ Demonstrate how to process **command-line arguments** in method **main**.
- ▶ static methods of class **Arrays** from the java.util package.
- ▶ **ArrayLists** are similar to arrays but provide additional functionality,



2.Arrays

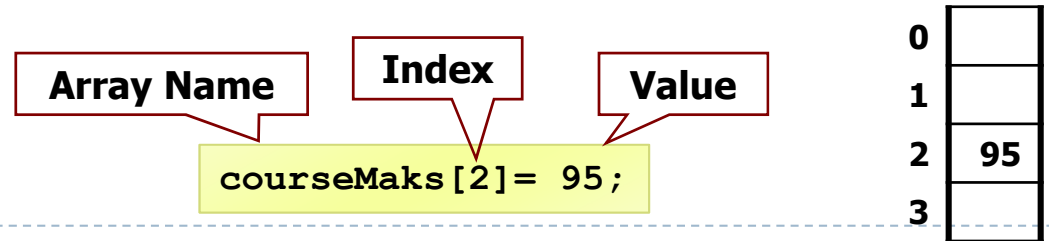
```
int[] ages = new int[10];  
String[] names = new String[10];
```

- ▶ An array is a group of variables (called elements) containing values that all have the **same** type.
- ▶ The **type** of an array is determined by its element type, which is specified when the array is instantiated.
- ▶ Arrays are **objects** and thus are of a **reference** type.
 - ▶ Elements can be either primitive types or reference types.
- ▶ Default values: (int:0; String: null; boolean:false)
- ▶ The length of an array can be obtained from the array object's **length** variable
 - ▶ An array's **length is fixed** when it is created (determines the amount of memory allocated) and cannot be changed unless the array is redeclared.

```
System.out.println(names.length);
```



2.Arrays



- ▶ Each element is accessed in constant time using an index position of the particular element in square brackets ([]).
- ▶ The index:
 - ▶ must be an int (non-negative), or an expression that evaluates to an int
 - ▶ uses zero-numbering (indexes go from 0 to length-1)
- ▶ Array names follow the same conventions as other variable names.
- ▶ Value (element of the array) can be accessed and modified in constant time using its index.



2.Arrays

Declaring & Creating Arrays

▶ The three steps for creating an array are

▶ Declaring an Array

- ▶ Syntax: `<type><name>[]; //or
<type>[] <name>;`

```
int courseMarks[];  
String[] args;
```

 `int[4] courseMarks;`

Specify the size in array declaration is a syntax error

- ▶ No memory space allocated
- ▶ No length specified
- ▶ The array has not been initialized

▶ Creating a New Array

- ▶ Syntax: `<name> = new <type>[Length];`

```
courseMarks = new int[10];
```

- specify the type of the array elements and the number of elements as part of an array-creation expression that uses keyword new.
- returns a reference that can be stored in an array variable.
- Note: memory space allocated, length specified, initialized to default values

▶ Creating & Initializing elements

- ▶ create an array and initialize its elements with an array initializer — a comma-separated list of expressions enclosed in braces.
- ▶ i.e. counts the number of initializers in the list to determine the size of the array
- ▶ Syntax:

```
<name> = new <type> { values };
```

```
courseMarks = new int[] {26,73,55,97};
```



2.Arrays

Creating Arrays

- ▶ Combine declaration and creation

- ▶ The first two steps of creating an array can be combined as one:

```
int[] courseMarks = new int[10];
```

- ▶ Combine declaration & initialization

- ▶ Array elements can be initialized in the declaration statement by putting a comma-separated list in braces
 - ▶ The length of an array is automatically determined when the values are explicitly initialized in the declaration

```
double[] reading = {5.1, 3.02, 9.65};
```

```
int[] courseMarks = new int[4]{26,73,55,97};
```



- ▶ The memory space of an array can be allocated dynamically during the run-time of the program.

- ▶ If the size is -1, it will lead to a run-time error
 - ▶ Note: The Java compiler will NOT indicate that something is wrong.

```
int[] courseMarks = new int[-1];
```





2.Arrays

Using Arrays

- ▶ Array processing is easily done in a loop
 - ▶ A for loop is commonly used to initialize array elements and print out all values

- ▶ Initialization

```
int[] reading = new int[5];
for (int i=0; i<reading.length; i++) {
    reading[i] = i;
}
```

- ▶ Note:

- ▶ The loop counter/array index goes from 0 to length-1
- ▶ It counts through length=5 iterations using the zero-numbering of the array index

- ▶ Printing

```
for (int i=0; i<reading.length; i++) {
    System.out.println(reading[i]);
}
```

- ▶ Using an index larger than length-1 causes a run time (not a compiler) error

- ▶ An `ArrayIndexOutOfBoundsException` is thrown

Outside the
valid range

```
System.out.println(reading[5]);
```



2.Arrays

Example 1

L06Code.java

► Example 1:

- creates a 10-element array and assigns to each element one of the even integers from 2 to 20 (2, 4, 6, ..., 20).
- `ARRAY_LENGTH`: is a constant.
 - Constant variables must be initialized before they're used and cannot be modified thereafter.

```
final int ARRAY_LENGTH = 5;
int[] array = new int[ARRAY_LENGTH];

for (int counter=0; counter<array.length; counter++)
    array[counter] = 2 + 2 * counter;

for (int counter=0; counter<array.length; counter++)
    System.out.printf("%5d%8d\n", counter, array[counter]);
```

0	2
1	4
2	6
3	8
4	10



2.Arrays

Enhanced for Statement

- Iterates through the elements of an array without using a counter, thus avoiding the possibility of “**stepping outside**” the array.
- Syntax:**

```
for (parameter : arrayName)
    statement
```

 - where parameter has a type and an identifier, and arrayName is the array through which to iterate.
- It can be used only to obtain array elements—it cannot be used to **modify** elements.
 - Note: No error message generated, the values are simply unchanged
 - If you need to modify elements, use the standard for loop

```
for (int value: array)
    System.out.println(value);
```

```
int[] array = new int[]{2, 4, 6, 8, 10};
for (int value: array)
    value += 2;
for (int value: array)
    System.out.printf("%5d\n", value);
```

```
2
4
6
8
10
```



Exercise 1-2

- ▶ Returns the number of even integers in the parameter array.

```
public static int countEvens(int[] numbers) {  
  
}
```

```
System.out.println(countEvens(new  
int[] {2, 4, 16, 80}));
```

4

- ▶ Returns the number of days in a specified month using the following month array.

```
int[] numberOfDays = new int[]{31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}
```

```
public static int getNumberOfDays(int month) {  
  
}
```

```
System.out.println(getNumberOfDays(12));
```

31



2.Arrays

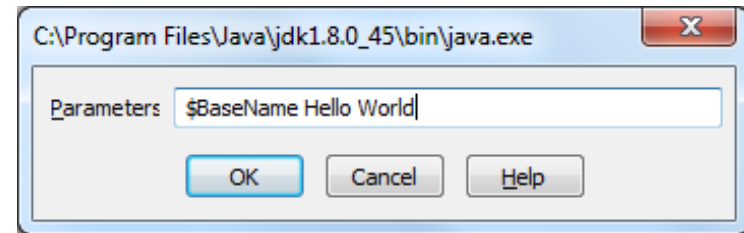
Arguments for the method `main`

- ▶ The heading for the main method shows a parameter that is an array of Strings:

```
public static void main(String[] args) {  
    ...  
}
```

- ▶ When you run a program from the command line, all words after the class name will be passed to the *main* method in the args array.

```
>java TestArray Hello World
```



- ▶ The following main method in the class *TestArray* will print out the first two arguments it receives:

```
public static void main(String[] args) {  
    for (int i=0; i<args.length; i++)  
        System.out.println(args[i]);  
}
```

Output:

**Hello
World**

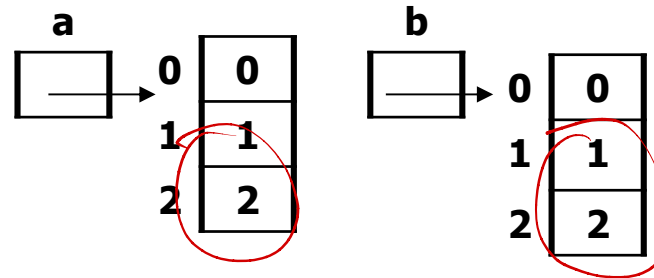


2.Arrays

Testing for Equality

- ▶ The following example creates two arrays of ints.

```
int[] a = new int[] {0, 1, 2};  
int[] b = new int[] {0, 1, 2};
```



- ▶ Comparison "=="

- ▶ Compares references, not values. The use of == with array/object references is generally limited to if two references are to the same object

```
if (b == a)  
    System.out.println("Equal");  
else  
    System.out.println("Not Equal");
```

Not Equal

*cannot use "==" between
Reference types.*

- ▶ Variables a and b are both 3-element arrays of ints
- ▶ The output of this example is "Not Equal" because the addresses of the arrays are not equal.

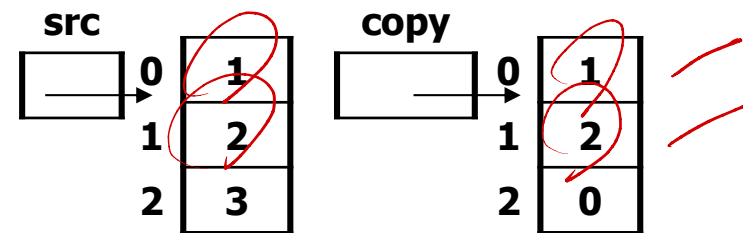


2.Arrays

Copying Arrays

- ▶ Note: assigning one array to another only changes the references. No actual copying is performed.
- ▶ To perform copying, use the
 - ▶ `void arraycopy(Object src, int srcPos, Object dest, int destPos, int length)`
 - ▶ Copies an array from the specified source array, beginning at the specified position, to the specified position of the destination array.

```
int[] src = {1,2,3};  
int[] copy = new int[3];  
System.arraycopy(src, 0, copy, 0, 2);  
for (int i=0; i<copy.length; i++)  
    System.out.println(copy[i]);
```



```
if (copy == src) //checking reference  
    System.out.println("Equal");  
else  
    System.out.println("Not Equal");
```

- ▶ Note:
 - ▶ It is useful when copying parts of an array
 - ▶ The copy array must be first created.

Not Equal



Exercise 3-4

- Increments each element value by one. Fix the problem

```
public static void incrementByOne(int[] numbers) {  
    for (int value: numbers)  
        value += 1;  
}
```



```
int[] numbers = {0, 1, 2};  
incrementByOne(numbers);  
...
```

[0, 1, 2]

- Takes an array of integers as a parameter
- Returns a **new** array containing the squared value of each integer in the parameter array.

```
int[] numbers = {1, 2, 3};
```

```
public static int[] getArrayOfSquares(int[] numbers) {  
  
  
}
```

[1, 4 9]



2.Arrays

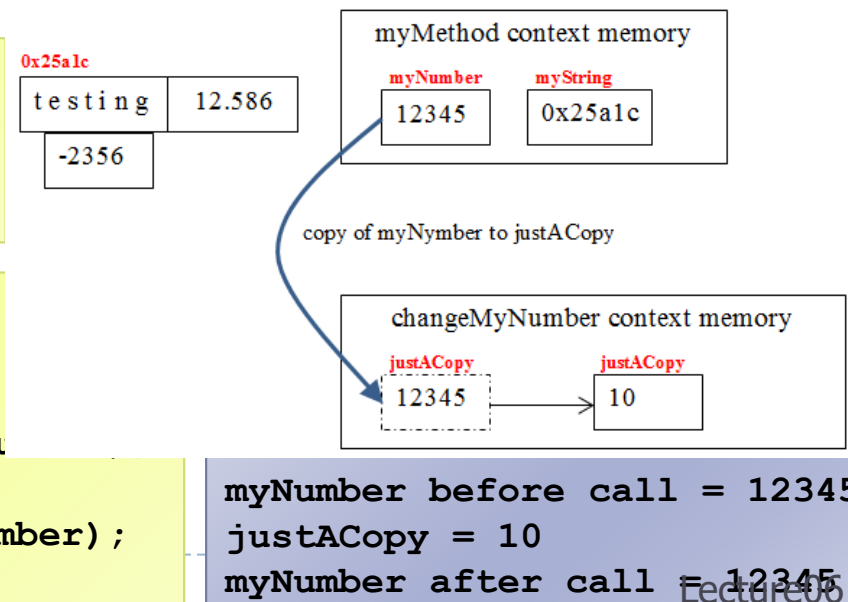
Pass-by-value

L06Code02.java

- ▶ All arguments are **passed by value** in method calls in Java.
- ▶ A method call can pass **TWO** types of values to a method—copies of **primitive values** and copies of **references** to objects.
 - ▶ Objects themselves cannot be passed to methods.
- ▶ Case 1: primitive types: (int, double, float.. etc)
 - ▶ a copy of the argument's value is passed to the called method.
 - ▶ changes to the called method's copy do not affect the original variable's value in the caller.

```
public static void changeMyNumber(int justACopy) {  
    justACopy = 10;  
    System.out.println("justACopy = "+justACopy);  
}
```

```
public static void myMethod() {  
    int myNumber = 12345;  
    String myString = "testing";  
    System.out.println("myNumber before call = "+myNu  
    changeMyNumber(myNumber);  
    System.out.println("myNumber after call = "+myNumber);  
}
```



2.Arrays

Pass-By-Value

L06Code02.java

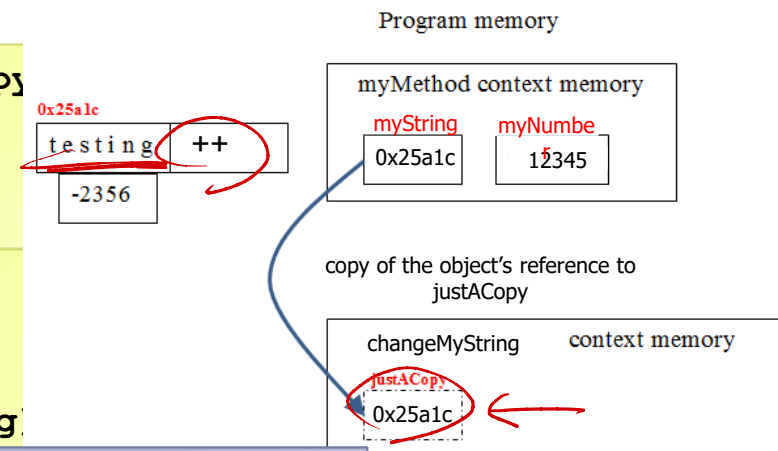
- ▶ Case 2: Reference Types (StringBuffer, array... etc)
 - ▶ i.e. a copy the object's reference is passed to the called method
 - ▶ an object's reference is passed by value, a method can still **interact** with the referenced object by calling its public methods using the copy of the object's reference.
 - ▶ The parameter in the called method and the argument in the calling method refer to the same object in memory.
 - ▶ We can modify the object and have the changes reflected in the outer scope.

```
public static void changeMyString(StringBuffer justACopy) {
    justACopy.append("++");
    System.out.println("justACopy = "+ justACopy);
}
```

```
public static void myMethod2() {
    int myNumber = 12345;
    StringBuffer myString = new StringBuffer("testing");
    System.out.println("myString before call = "+myString);
    changeMyString(myString);
    System.out.println("myString after call = "+myString);
}
```

18

myString before call = testing
justACopy = testing++
myString after call = testing++



Lecture06



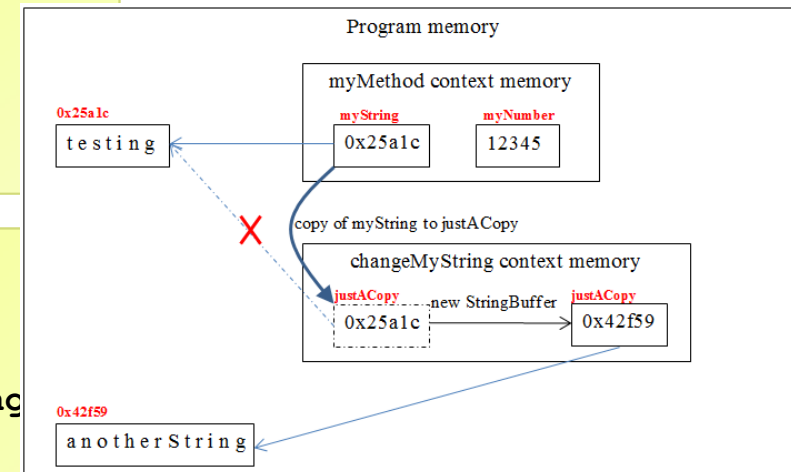
2.Arrays Pass-By-Value

L06Code02.java

- ▶ Case 3: Reference Types (StringBuffer, array... etc)
 - ▶ i.e. a copy the object's reference is passed to the called method
 - ▶ If you **modify** a reference-type parameter so that it refers to **another object**, only the parameter refers to the new one.
 - ▶ changes to the called method's copy do not affect the original variable's value in the caller.

```
public static void makeNewString(StringBuffer justACopy) {  
    justACopy = new StringBuffer("another String");  
  
    System.out.println("justACopy = "+justACopy);  
}
```

```
public static void myMethod3() {  
    int myNumber = 12345;  
    StringBuffer myString = new StringBuffer("testing");  
    System.out.println("myString before call = "+myString);  
    makeNewString(myString);  
    System.out.println("myString after call = "+myString);  
}
```



myString before call = testing
justACopy = another String
myString after call = testing



2.Arrays

Passing Arrays to Methods

- ▶ To pass an array argument to a method, specify the **name** of the array without any brackets.

```
modifyArray(origArray);
```

- ▶ the method's parameter list must specify an **array parameter**.

```
public static void modifyArray(int[] myArray) { ... }
```

- ▶ the called method receives a **copy of the object's reference**.
 - ▶ i.e. they both refer to the same object
- ▶ We can modify the object and have the changes reflected in the outer scope.

```
public static void modifyArray(int[] myArray) {  
    myArray[0] += 10;  
}
```

```
origArray before call = 3  
origArray after call = 13
```

```
int[] origArray = {3,5,7};  
System.out.println("origArray[0] before call = "+origArray[0]);  
modifyArray(origArray);  
System.out.println("origArray[0] after call = "+origArray[0]);
```



2.Arrays

Passing Array elements to Methods

- ▶ To pass an individual array element of a primitive type to a method, specify the **name** of the array and the index.

```
changeMyNumber(origArray[2]);
```

- ▶ the called method receives a copy of the element's value.
- ▶ Modifying the copy in the called method does **not** affect the original value of that element in the calling method's array.

```
public static void changeMyNumber(int justACopy){  
    justACopy = 10;  
    System.out.println("justACopy = "+justACopy);  
}
```

```
int[] origArray = {3,5,7};  
System.out.println("origArray[2] before call = "+origArray[2]);  
changeMyNumber(origArray[2]);  
System.out.println("origArray[2] after call = "+origArray[2]);
```

```
origArray[2] before call = 7  
justACopy = 10  
origArray[2] after call = 7
```



2.Arrays

Variable-Length Argument Lists

- ▶ With **variable-length** argument lists, you can create methods that receive an **unspecified** number of arguments.
 - ▶ Syntax: A type followed by an ellipsis (...) in a method's parameter list
 - ▶ e.g. double... numbers
- ▶ Note: it can occur only **once** in a parameter list, and must be placed at the **end** of the parameter list.
- ▶ Java treats the variable-length argument list as an array whose elements are all of the same type.

```
public static double average(double... numbers) {  
    double total = 0;  
    for (double d: numbers)  
        total += d;  
    return total/numbers.length;  
}
```

```
Average 15.0  
Average 20.0
```

```
System.out.printf("Average %.1f%n", average(d1, d2));  
System.out.printf("Average %.1f%n", average(d1, d2, d3));
```



Exercise 5

- The output of the following code fragment:

```
int[] array = {0, 2, 4, 6, 8};  
System.out.println(Arrays.toString(array));
```

```
[0, 2, 4, 6, 8]
```

Consider the following static method:

```
public static void changeArray(int[] passedArray, int value){  
    passedArray[value] = 12;  
    value = 5;  
}
```

- What is the output of the following code fragment?

```
changeArray(array, array[1]);  
System.out.println(Arrays.toString(array));
```

```
[0, 2, ...]
```



3. Multidimensional Arrays

- ▶ **Multidimensional Arrays** - Arrays with more than one index
 - ▶ number of dimensions = number of indexes
- ▶ Multidimensional arrays with two dimensions are often used to represent tables of values with data arranged in rows and columns.
 - ▶ To identify a particular table element, you specify two indices—the first identifies the element's row and the second its column.
- ▶ Multidimensional arrays can have more than two dimensions.

```
int[][] x = new int[5][7];
```

```
int[][][] threeD = new int [2][3][4];  
int[][][][] fourD = new int [2][3][768][1024];
```

5 rows

7 columns:



3. Multidimensional Arrays

2-Dimensional Arrays

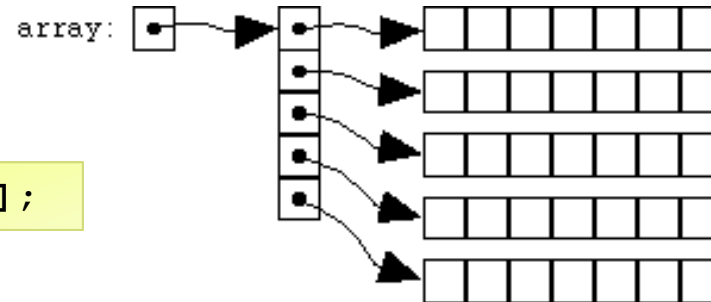
▶ Declaration & Initialization 2-D arrays

- ▶ Syntax for 2-D arrays is similar to 1-D arrays
 - ▶ `<type>[][] <name> = new <type>[rows][columns];`

▶ Example 1:

```
int[][] x = new int[5][7];
```

- ▶ The array should have five rows and seven columns
- ▶ The real picture:



▶ Example 2:

```
int[][] x = new int [4][];
```

- ▶ The array has 4 rows.

▶ 2-D Array_INITIALIZER

- ▶ You can also initialize at declaration with 2D array literal
 - ▶ Use nested comma separated lists, enclosed in {}'s

```
int[][] y = { {1,2,3}, {4,5,6}};
```

1	2	3
4	5	6

Compiler determine the number of rows and determine the number of columns in each row

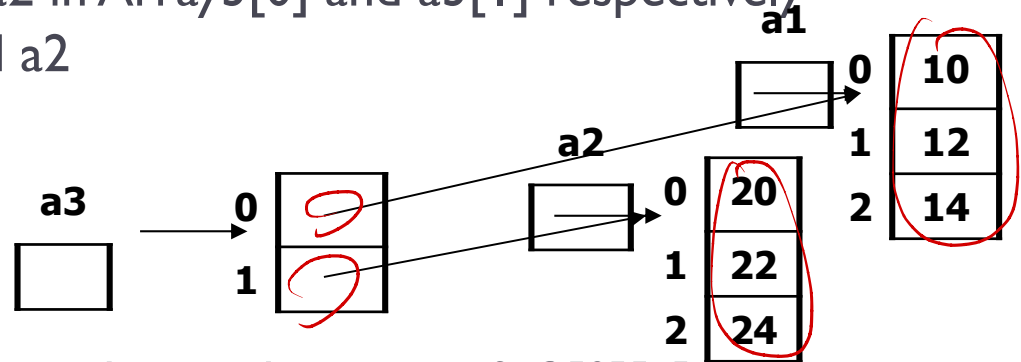


3. Multidimensional Arrays

Creating Multidimensional Arrays

- ▶ We can use existing arrays for initialization
 - ▶ a3 does not store copies of a1 and a2 in Array3[0] and a3[1] respectively
 - ▶ Instead it stores reference to a1 and a2

```
int[] a1 = {10,12,14};  
int[] a2 = {20,22,24};  
int[][] a3 = {a1, a2};
```

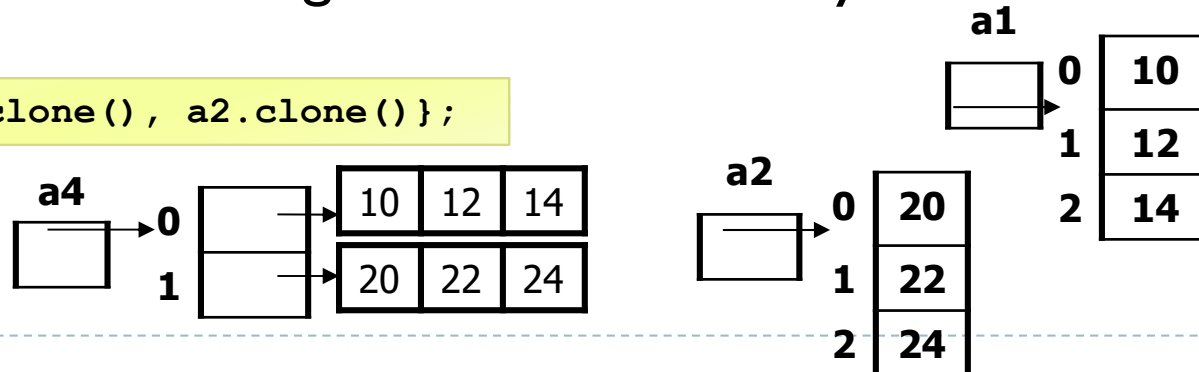


- ▶ If we change the value of a1[1], we also change the value of a3[0][1]

```
a1[1] = 0;  
System.out.println(a3[0][1]);
```

~~✱~~ To create copies of an existing 1-dimensional arrays, we can use the clone method

```
int[][] a4={a1.clone(), a2.clone()};
```





3. Multidimensional Arrays

Length Variables

► Using the Length Variable:

► arrayName.length

- returns the number of rows in the array

► arrayName[RowIndex].length

- returns the number of columns in row "RowIndex"

```
int[][] (x) = new int[5][7];  
System.out.println(x.length);  
System.out.println(x[0].length);
```

► Note:

- If x is a rectangular array, all rows have the same length

```
System.out.println(x[0].length);  
System.out.println(x[1].length);  
System.out.println(x[2].length);
```



3. Multidimensional Arrays Using 2D Arrays

L06Code03.java

► Accessing elements

- To access elements, we use pretty much the same technique as a 1D array

```
System.out.println(myArr[rowToGet][colToGet]);
```

► Initialization & Printing

- Nested loops are used to initialize array elements and print out values
- Starting from the first row (0th) to the last row (n-1) – outer loop
 - For each row, starting from the first column (0th) to the last column (m-1) – inner loop

Use `y.length` to
navigate the last row

```
int[][] y = { {1,2,3}, {4,5,6} },
for (int row = 0; row < y.length; row++) {
    for (int col = 0; col < y[row].length; col++)
        System.out.print(y[row][col] + " ");
    System.out.println();
}
```

Use `y[row].length` to navigate
the last column of each row

Or using for-enhanced
loop

```
for (int[] array: numbers) {
    for (int number : array)
        System.out.print(number + " ");
    System.out.println();
}
```



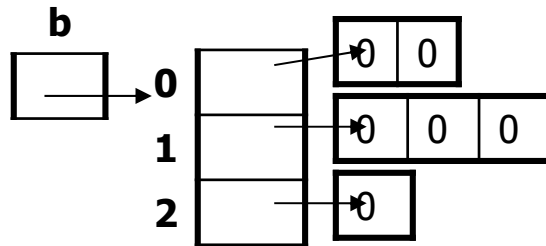
3. Multidimensional Arrays

Ragged Arrays

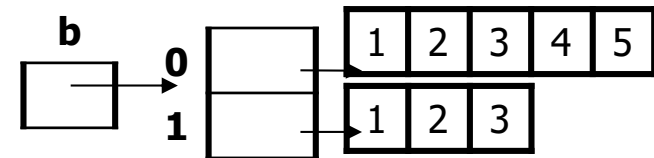
- ▶ Ragged arrays have rows of unequal length
- ▶ Each row has a different number of columns, or entries
- ▶ Declaration & Initialization a ragged array

- ▶ `<type>[][] <name> = new <type>[rows][ ];`
- ▶ `<name>[0] = new <type>[colCountForRow1];`
- ▶ ...

```
int[][] b;  
b = new int[3][ ];  
b[0] = new int[2];  
b[1] = new int[3];  
b[2] = new int[1];
```

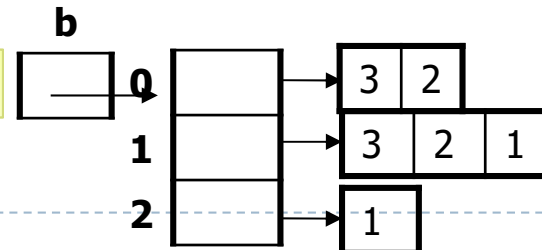


```
int[][] b1 = new int[2][ ];  
b1[0] = new int[]{1, 2, 3, 4, 5};  
b1[1] = new int[]{1, 2, 3};
```



- ▶ Using Array Initializer

```
int[][] b = {{3,2}, {3,2,1}, {1}};
```





3. Multidimensional Arrays

Using Ragged Arrays

▶ Length Variables:

- ▶ `arrayName.length` returns the number of rows in the array
- ▶ `arrayName[rowToGet].length` returns the number of columns in row "rowToGet"

```
int[][] y = {{3,2}, {3,2,1}, {1}};
```

▶ Example:

- ▶ `y.length = 3`
- ▶ `y[0].length` returns 2
- ▶ `y[1].length` returns 3
- ▶ `y[2].length` returns 1
- ▶ `y[3].length` <- ERROR

▶ Initialization & Printing

- ▶ Nested loops are used to initialize array elements and print out values
- ▶ Note: You must use `y[row].length` to get the number of columns in each row. The number may not be the same for all rows.

```
for (int row = 0; row < y.length; row++) {  
    for (int col = 0; col < y[row].length; col++)  
        System.out.print(y[row][col] + " ");  
    System.out.println();  
}
```

Exercise 6

```
int[][] grid = { { 3, 5, 6, 9 }, { 2, 6, 9, 4 } };  
int len;
```

- ▶ What is the output of the following code fragment?

```
len = grid[0].length;  
for(int row=0; row < len; row++) {  
    for(int col=0; col < len; col++) {  
        System.out.print(grid[row][col] + " ");  
    }  
    System.out.println();  
}
```



```
len = grid.length;  
for(int row=0; row < len; row++) {  
    for(int col=0; col < len; col++) {  
        System.out.print(grid[row][col] + " ");  
    }  
    System.out.println();  
}
```

- ▶ Q6:

- ▶ Takes an array of integer arrays and an integer
- ▶ Returns the sum of all entries in the parameter array of arrays which are greater than a specified amount, target.

```
public static int sumOver(int[][] values, int target) {
```

```
}
```

```
int[][] numbers = {{2, 4, 16, 80, 27}, {1, 4, 100,  
18, 7}, {20, 14, 70, 8, 130}};
```

```
System.out.println(sumOver(numbers, 90));
```

230



4. Array of Objects

- ▶ Apart from arrays of primitive types we can have arrays of reference types (objects)

- ▶ Creating an array of Point

- ▶ Syntax:

- ▶ Create the array

- `<class>[] <name> = new <class>[Length];`

- `Point[] p = new Point[3];`

- Note: Memory space are allocated for the array

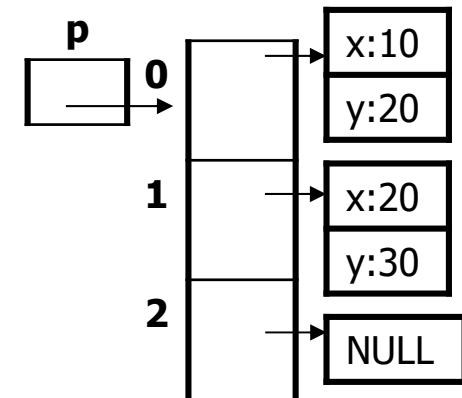
- ▶ Create the object element

- `<name>[row] = new <class>();`

- `p[0] = new Point(10, 20);`
`p[1] = new Point(20, 30);`

- ▶ Note:

- ▶ In an array of reference types not the objects are stored in the array, but references to the objects.
 - ▶ The objects themselves have to be created and memory space has to be allocated for them separately.
 - ▶ The element are initialized with the null reference.





4. Array of Objects

Using Array of Objects

► Accessing

- To access all components of an array of reference types we can use the same code that we have used for primitive types
 - A for loop is used to print out values.

```
for (int i=0; i<2; i++)  
    System.out.println(p[i].toString());
```

```
java.awt.Point[x=10,y=20]  
java.awt.Point[x=20,y=30]
```

- You must create the object element before accessing it.
 - Otherwise, a `NullPointerException` is thrown.

```
for (int i=0; i<p.length; i++)  
    System.out.println("(" + p[i].x + "," + p[i].y + ")");
```

```
(10,20)  
(20,30)  
Exception in thread "main" java.lang.NullPointerException ...
```

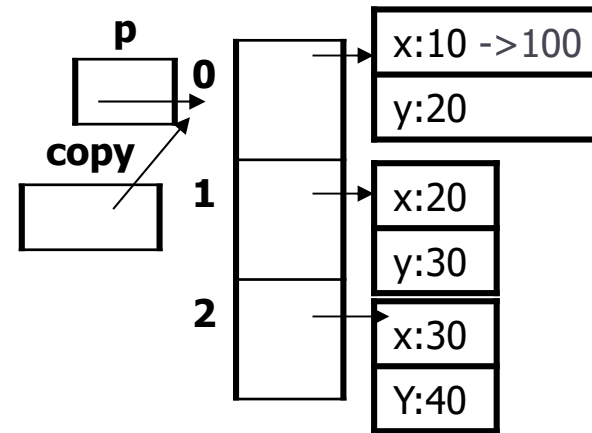


4. Array of Objects

Assignment - Arrays of Objects

Reference type:

```
Point[] p = new Point[3];  
p[0] = new Point(10, 20);  
p[1] = new Point(20, 30);  
p[2] = new Point(30, 40);  
  
Point[] copy = p;
```



- ▶ The variable `copy` holds a copy of the reference held in the variable `p`. There is still only one copy of the array.
- ▶ Thus changing the object pointed to by one of the variables will also cause the contents of the other variable to change (since the same object in memory is being altered).

```
copy[0].x = 100;  
for (int i=0; i<copy.length; i++)  
    System.out.println(p[i].toString());
```

```
java.awt.Point[x=100,y=20]  
java.awt.Point[x=20,y=30]  
java.awt.Point[x=30,y=40]
```



Exercise 7

- ▶ Takes an array of strings as a parameter
- ▶ Returns the number of words in the array starting with a vowel.

```
public static int startWithVowelCount(String[] words) {  
  
}
```

1

```
String[] array = new String[]{"hello",  
"good", "bad", "oh"};  
System.out.println(startWithVowelCount(ar  
ray));
```



5. ArrayList

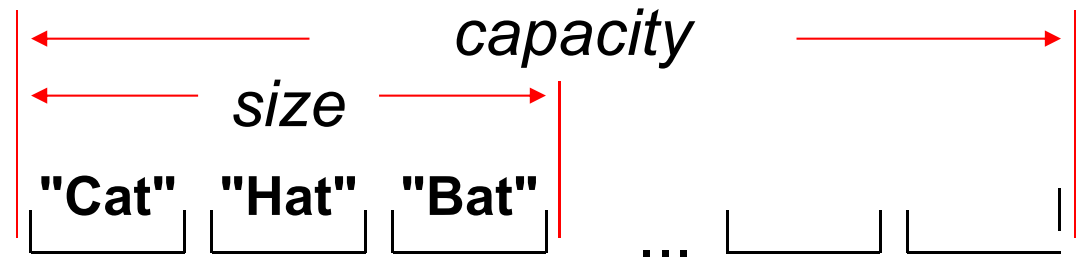
```
import java.util.ArrayList;
```

- ▶ Introduction to Collections and Class ArrayList
- ▶ Collections provide efficient methods that organize, store and retrieve your data without requiring knowledge of how the data is being stored.

```
ArrayList<String> list = new ArrayList<String>();  
list.add("apple");
```

- ▶ ArrayLists:

- ▶ Everything stored must be an Object (non-primitive data types)
- ▶ provides methods for adding and removing
- ▶ keeps track of the list capacity (the length of the allocated array) and list size (the number of elements currently in the list)



```
ArrayList<Integer> nums = new ArrayList<Integer>();
```



2. ArrayLists Methods

Method	Description
<code>add</code>	Adds an element to the <i>end</i> of the <code>ArrayList</code> .
<code>clear</code>	Removes all the elements from the <code>ArrayList</code> .
<code>contains</code>	Returns <code>true</code> if the <code>ArrayList</code> contains the specified element; otherwise, returns <code>false</code> .
<code>get</code>	Returns the element at the specified index.
<code>indexOf</code>	Returns the index of the first occurrence of the specified element in the <code>ArrayList</code> .
<code>remove</code>	Overloaded. Removes the first occurrence of the specified value or the element at the specified index.
<code>size</code>	Returns the number of elements stored in the <code>ArrayList</code> .
<code>trimToSize</code>	Trims the capacity of the <code>ArrayList</code> to the current number of elements.



2. ArrayLists

Methods to add elements

- ▶ Only objects can be added to an ArrayList.

- ▶ `public boolean add(E x)`

```
ArrayList<String> list = new ArrayList<String>();
```

**E is the type you specified
when creating the ArrayList
i.e. String (in our example)**

```
list.add( "One" );  
list.add( "Two" );  
list.add( "Three" );
```

One	Two	Three
-----	-----	-------

- ▶ Adds an object to the end of the list, adjusts the size of the list; returns true

- ▶ Objects can be added to a specific position in an ArrayList:

- ▶ `public void add(int index, E o)`

```
list.add( "One" );  
list.add( "Two" );  
list.add( "Three" );  
list.add( 1, "Fred" );
```

One	Fred	Two	Three
-----	------	-----	-------

- ▶ Inserts x at position index, sliding elements at position index and higher to the right (adds 1 to their indices) and adjusts size.

- ▶ Capacity increases if necessary



2. ArrayLists

Search methods

- ▶ **public boolean contains (E target)**

- ▶ True if ArrayList contains target; false otherwise.

```
System.out.println(list.contains("Three" ));
```

true

- ▶ **public int indexOf (E target)**

- ▶ Returns index of first occurrence of target in ArrayList; -1 otherwise.

```
int i = list.indexOf("Fred" );
```

1

- ▶ **public int lastIndexOf (E target)**

- ▶ Same as above except index of last occurrence is returned.

- ▶ **public int size ()**

```
System.out.println(list.size());
```

- ▶ Returns number of elements in ArrayList

4



2. ArrayLists Size & Capacity

- ▶ `public boolean isEmpty ( )`
 - ▶ True if empty; false otherwise.

```
System.out.println(list.isEmpty());
```

```
false
```

- ▶ `public void clear ( )`
 - ▶ Removes all elements; `size()` becomes 0

```
list.clear();
```

- ▶ `public String toString()`
 - ▶ Returns a String which contains all the object in the ArrayList

```
System.out.println(list);
```

```
System.out.println(list.toString());
```

```
[One, Fred, Two, Three]
```

- ▶ Using a For loop

```
for( int i=0; i<list.size(); i++ ) {  
    System.out.println(list.get(i).toString());  
}
```

```
One  
Fred  
Two  
Three
```

```
for (String value: list)  
    System.out.println(value);
```




2.ArrayLists

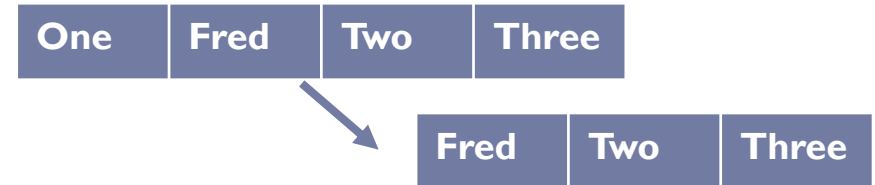
Methods to remove elements

▶ remove(int index)

- ▶ where $0 \leq \text{index} < \text{size}()$ (or exception)
- ▶ Removes element at index; shifts to the left remaining elements at $\text{index}+1 \dots \text{size}()-1$.

```
list.remove( 0 );
```

```
String s = list.remove( 0 );  
System.out.println(s);
```



One

▶ boolean remove(T theElement)

- ▶ if found then removes the first occurrence of theElement; shifts the remaining elements to the left; $\text{size}()$ becomes $\text{size}()-1$; returns true.
- ▶ if not found then returns false.

```
boolean n = list.remove("Three");  
System.out.println(n);
```

One Fred Two Three

true

One Fred Two



2. ArrayLists

Array-like methods

- ▶ `public E set ( int index, E newElement )`
 - ▶ where $0 \leq \text{index} < \text{size}()$ (or exception)
 - ▶ Replaces the element at index with the parameter and returns the element formerly at the specified position.

```
System.out.println(list.set(0, "Hello"));
```

One

One	Fred	Two
-----	------	-----

Hello	Fred	Two
-------	------	-----

```
System.out.println(list.set(4, "Good"));
```



Exception in thread "main"
`java.lang.IndexOutOfBoundsException:`

- ▶ `public E get ( int index )`
 - ▶ where $0 \leq \text{index} < \text{size}()$ (or exception)
 - ▶ Returns the element at index.

```
System.out.println(list.get(0));
```

Hello

Hello	Fred	Two
-------	------	-----

```
System.out.println(list.get(4));
```



Exception in thread "main"
`java.lang.IndexOutOfBoundsException:`



2. ArrayLists Equality

- ▶ `public boolean equals ( Object other )`
 - ▶ True only when both are of the **same size**, and both have **the same elements with the same order**.

```
list.add("Hello");  
list.add("Two");  
list.add("Fred");  
  
myList2.add("Two");  
myList2.add("Hello");  
myList2.add("Fred");  
System.out.println(list.equals(myList2));
```

false

5. ArrayLists (Generics)

```
ArrayList words = new ArrayList();
```



OLD version

- ▶ Starting with Java 5, `ArrayList<E>` and other collection classes hold objects of **a** specified data type.
 - ▶ The `E` (by convention) is a placeholder—when declaring a new `ArrayList` with the **type** of elements that you want the `ArrayList` to hold.
 - ▶ Classes with this kind of placeholder that can be used with any type are called **generic classes**. For examples:

```
ArrayList<String> words = new ArrayList<String>();  
ArrayList<Integer> nums = new ArrayList<Integer>();
```

- ▶ Using `<>` in a class instance creation expression tells the compiler to **determine** what belongs in the angle brackets
- ▶ `ArrayList<String>` appears in the **variable declaration** and in the **class instance creation** expression.

```
ArrayList<String> words = new ArrayList<String>();  
words.add("One");  
words.add(new Point(10,20));
```





4. Wrapper Classes

- ▶ Each primitive data type has a corresponding Wrapper class.
 - ▶ Java's primitive data types (boolean, int, etc.) are not classes.
 - ▶ Wrapper classes are used in situations where objects are required.
 - ▶ The wrapper class can be used to convert a primitive into an object type.

Data Type	Wrapper
boolean	Boolean
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double



4. Wrapper Classes

Value => Object: Wrapper Object Creation

- ▶ `Wrapper.valueOf()` takes a value (or string) and returns an object of that class:

```
Integer i1 = Integer.valueOf(42);  
Integer i2 = Integer.valueOf("42");  
  
Boolean b1 = Boolean.valueOf(true);  
Boolean b2 = Boolean.valueOf("true");
```

int → Integer

- ▶ Using object creation:

```
Integer i1 = new Integer(42);  
  
Character cC = new Character('b');  
  
Boolean bB = new Boolean(true);
```



4. Wrapper Classes

Object => Value & String => value

- Each wrapper class Type has a method *typeValue* to obtain the object's value: `intValue()`, `doubleValue()`, `charValue()` etc

```
System.out.println(i1.intValue());  
System.out.println(c1.charValue());  
System.out.println(b1.booleanValue());
```

```
42  
b  
true
```

integer → int

- It also has a method `parseType()` to parse a string representation & return the literal value.

```
Integer.parseInt("42")           => 42  
Boolean.parseBoolean("true")     => true  
Double.parseDouble("2.71")       => 2.71
```



4. Wrapper Classes

Auto boxing

- ▶ Since Java 5, conversion from int to Integer and from double to Double is, in most cases, automatic (autoboxing/autounboxing)

```
ArrayList<Integer> numbers = new ArrayList<Integer>();  
  
numbers.add(12);  
numbers.add(5);  
int sum = 0;  
  
for (int i : numbers) {  
    sum += i;  
}  
System.out.println("Total=" + sum);
```

int → Integer (automatic)

autoboxing

autounboxing

same here



Exercise 8-9

- ▶ What is the output of the following code fragment?

```
ArrayList<Point> list = new ArrayList<Point> ();  
Point pt1 = new Point(3, 4);  
list.add( pt1 );  
Point pt2 = list.get( 0 );  
pt2.x = 23;  
System.out.println( pt1 == pt2 );
```

- ▶ What is the output of the following code fragment?

```
ArrayList<Integer> list1 = new ArrayList<Integer>();  
list1.add(3);  
list1.add(5);  
list1.add(8);  
list1.add(9);  
ArrayList<Integer> list2 = new ArrayList<Integer>();  
list2.add(6);  
list2.add(7);  
list2.add(1);  
list2.add(2);  
for (int i=0; i<list1.size(); i++) {  
    list1.set(i, list2.get(i));  
    list2.set(i, list1.get(i));  
}
```



- ```
public static void removePositives (ArrayList<Integer> numbers) {
```

$$[-34, -4]$$



## 5. ArrayLists Vs Arrays

### ► Capacity:

- Array: Once you set the array size, you cannot change it easily.

```
int actualSize = . . . ;
Point[] points1 = new Point[actualSize];
```

- ArrayList: It automatically adjusts its capacity as you add and remove elements, without your needing to write any code.

```
ArrayList<Point> points2 = new ArrayList<Point>();
```

Initial capacity = 10

- If the internal array is full, the array list automatically creates a bigger array and copies all the objects from the smaller to the bigger array.

```
ArrayList<Point> points2 = new ArrayList<Point>(100);
```

capacity = 100

### ► Printing the entire array/ArrayList:

- Array : print the memory address only

```
System.out.println(points1); [Ljava.awt.Point;@659e0bfd
```

- ArrayList: print the entire list

```
System.out.println(points2); [java.awt.Point[x=1,y=2],
java.awt.Point[x=3,y=4]]
```



## 5. ArrayLists Vs Arrays

### Number of elements & Accessing Elements

#### ▶ Actual number of elements inside the array/ArrayList

##### ▶ array:

```
points1.length
```

- ▶ returns the actual number of elements in the array

*string: str.length()*

##### ▶ ArrayList:

```
points2.size()
```

- ▶ returns the actual number of elements in the ArrayList

#### ▶ To change the element:

##### ▶ Array:

```
points1[i] = new Point(1,2);
```

- ▶ Use [] syntax to access or change the element of an array

##### ▶ ArrayList:

```
points2.set(i, new Point(1,2));
```

- ▶ Use the set method to change the element in the ArrayList

#### ▶ To get an element

##### ▶ Array:

```
Point p = points1[i];
```

- ▶ Use [] syntax to access or change the element of an array

##### ▶ ArrayList:

```
Point p = points2.get(i);
```

- ▶ Use the get method to access the element in the ArrayList



## 5. ArrayLists Vs Arrays

### Converting between Array and ArrayList

#### ► Array to ArrayList

- Use the `asList()` method in the `Arrays` class

```
Point[] points1 = {new Point(1,2), new Point(3,4)};
ArrayList<Point> points2 = new ArrayList<Point>(Arrays.asList(points1));
System.out.println(points2);
System.out.println(points1);
System.out.println(Arrays.toString(points1));
```

- Note: The `Arrays` class provides static methods for common array manipulations. For example:

- `sort()` - for sorting an array
- `toString()` returns a string

```
[java.awt.Point[x=1,y=2], java.awt.Point[x=3,y=4]]
[Ljava.awt.Point;@6bc7c054
[java.awt.Point[x=1,y=2], java.awt.Point[x=3,y=4]]
```

#### ► ArrayList to Array

- Use the `toArray()` method from the `ArrayList` class

```
String[] copy = new String[words.size()];
copy = words.toArray();
for (String i: copy)
 System.out.println(i);
```

```
One
Two
Three
```